

Тема 10. ОТСИЧАНЕ НА ГРАФИЧНИ ПРОЗОРЦИ, ЗАПЪЛВАНЕ НА ЗАТВОРЕНИ ОБЛАСТИ, ТЕКСТУРИ, ЗАМЪГЛЯВАНЕ И ПРОЗРАЧНОСТ

Алгоритми за запъване на затворени области и отсичане на графични прозорци. Използване на различни видове цветни текстури, стилове, маски, замъглявания, полупрозрачност и др. графични ефекти в средата на OpenGL.

упр. 12 Използване на различни атрибути на графичните примитиви (цветове, текстури, стилове, маски, мъгла, полупрозрачност и др.) в средата на OpenGL. OpenGL – OpenGL – Запъване с текстури, Замъглявания и Полу-прозрачност.

10.1. ПРОРЯЗВАНЕ НА КОНТУР

Често се изисква отделянето на фрагменти от изображението и акцентиране на вниманието върху него. Отделеният фрагмент може да се оцвети с друг цвят или да се огради с окръжност или правоъгълник. Отделянето на фрагменти от изображението с помощта на окръжности и правоъгълници е удобно да се използва в отчети или при обяснения на някакъв материал. Възможно е рамката да се движи, съпровождайки обясненията. Рамката се премества, отделяйки фрагменти от изображението, които ни интересуват. Рамката може да се задържи на едно място дотогава, докато сме готови да преминем към обсъждане на следващ фрагмент. По този начин е удобно да се отделят фрагменти, които са елементи на списък, част от графика или диаграма.

Изтриване и отсичане

Понякога се изисква да се изключи част от създаваното изображение. Тогава се използват методите за отделяне на области за идентификация на фрагментите от изображението, които трябва да се изтрият или отсекаат. В първия случай отделяме изображението в указаната област от екрана, а във втория съхраняваме изображението само в тази област и изтриваме всичко останало.

10.1.1. Изтриване

Методите за отделяне на области от екрана с окръжност или правоъгълник могат да се използват за определяне на областта за изтриване. Необходимостта за отделяне на части от изображението възниква поради много причини (например за опростяване на сложно изображение или освобождаване на място за засветяване на допълнителна информация).

Ако се изисква често отделяне на фрагменти, като например в задачите за проектиране, то се налага да се разработи програма за изтриване на информацията вътре в зададена област.

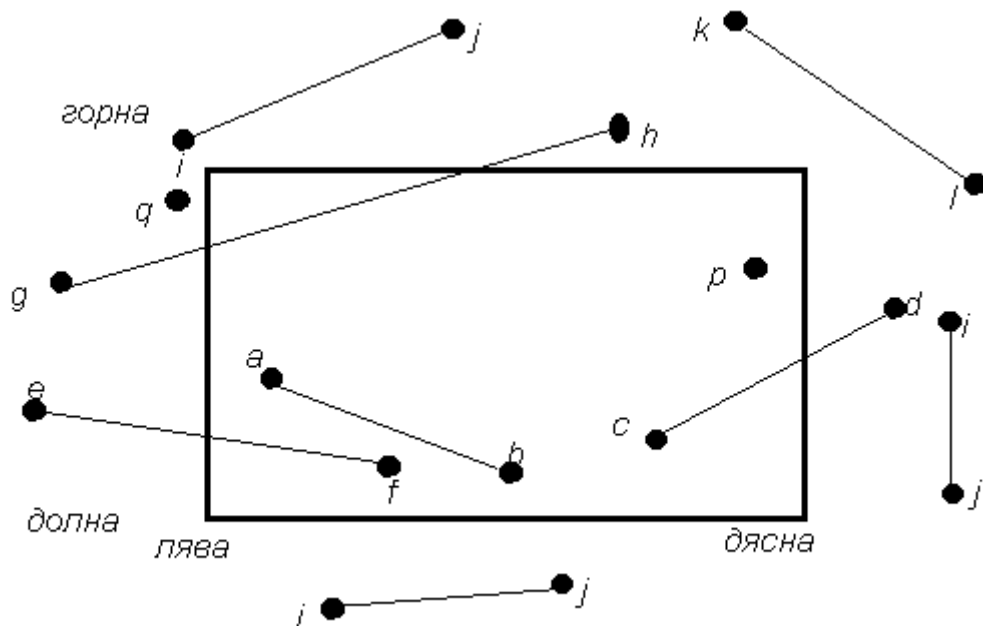
Използването на правоъгълник за идентификация на изчистваната област позволява леко да се изтрият символи, точки и части от редове, намиращи се вътре в правоъгълника: трябва да се запълни този правоъгълник с линии, цвета на които съвпада с цвета на фона на екрана.

Изчистването може да се изпълни и в границите на окръжност. В този случай това ще стане по-бавно, тъй като се изчисляват координатите на точките от окръжност, които след това се използват за указание на краищата на отсечки от линии, запълващи вътрешната област на окръжността.

10.1.2. Отсичане

В някои случаи се изисква да се съхрани част от изображението и всичко останало да се изчисти. Това възниква например, когато искаме да увеличим малка област от рисунката или графиката до размерите на целия екран или да получим изображение на отделни части от рисунката за отчет или илюстрация. За тази цел избраната област от екрана се огражда в правоъгълна рамка, наречена **графичен прозорец**.

Програмата, изпълняваща отсичането, трябва да намери и запази всички надписи, отделни точки и части от линии, попадащи в областта на прозореца.



Фиг. 10.1.1

На фиг. 10.1.1 е показана плоска сцена и отсичащ прозорец с форма на правилен четириъгълник. Прозорецът се задава с **ляво, дясно, горно и долно двумерни ребра**. **Правилният графичен прозорец** е правоъгълник, чиито страни са успоредни на координатните оси. Целта на алгоритъма за отсичане е да се определят онези **точки, отсечки или частите им**, които **лежат вътре** в отсичащия прозорец. Тези точки, отсечки или части от тях остават за визуализация, а останалите се изтриват.

Тъй като в обичайните сцени трябва да се отсека голям брой отсечки или точки, то особен интерес представлява ефективността на алгоритмите за отсичане. В много случаи отсечките лежат изцяло вътре или извън графичния прозорец. Затова е важно бързо да се отделят отсечките, подобни на *ab* или точки, подобни на *p* и да се отхвърлят отсечки като *ij* или точки като *q*. Точките, лежащи вътре в графичния прозорец удовлетворяват условието:

$$x_l \leq x \leq x_d \text{ и } y_d \leq y \leq y_e$$

където x_l , x_d , y_d , y_e са съответно 'лявата' и 'дясна' координата по x и 'долната' и 'горна' координата по y .

Точките на границата се считат за лежащи **вътре** в прозореца. **Отсечката** лежи **вътре** в прозореца и е **видима**, ако **двете и крайни точки лежат вътре** в прозореца. Ако **двата и края** лежат **извън** прозореца, то такава отсечка лежи **незадължително изцяло извън** прозореца, например отсечка *gh*. Ако и двата края лежат **отгоре, отдолу, отляво или отдясно**, то отсечката **изцяло лежи извън** прозореца. Проверката на последното условие отстранява всички отсечки, подобни на *ij*. Но тя не отстранява нито отсечка *qh*, която е частично видима, нито отсечка *kl*, която е изцяло невидима.

Алгоритъм, определящ изцяло видимите и повечето невидими отсечки [11]

a, b - крайни точки на отсечката в координатното пространство (x, y) .

За всяка отсечка се проверява пълната видимост. Ако една от координатите на който и да е край се намира извън прозореца, то отсечката не е напълно видима.

if $x_a < x_l$ **or** $x_a > x_d$ **then 1 endif**

if $x_b < x_l$ **or** $x_b > x_d$ **then 1 endif**

if $y_a < y_d$ **or** $y_a > y_e$ **then 1 endif**

if $y_b < y_d$ **or** $y_b > y_e$ **then 1 endif**

отсечката е с пълна видимост и тя се визуализира.

go to 3

проверява се пълната невидимост на отсечката. Ако двата края лежат отляво, отдясно, отгоре или отдолу на прозореца, отсечката е изцяло невидима.

1 if $x_a < x_l$ **and** $x_b < x_l$ **then 2 endif**

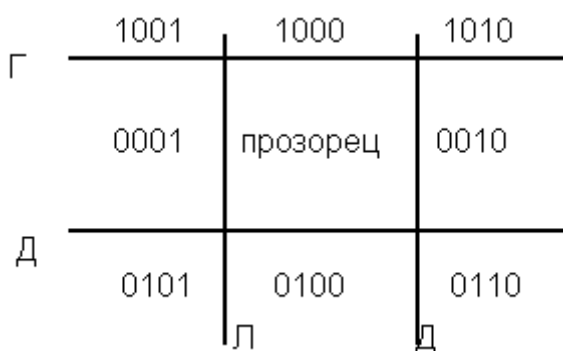
if $x_a > x_d$ **and** $x_b > x_d$ **then 2 endif**

if $y_a < y_d$ **and** $y_b < y_d$ **then 2 endif**

if $y_a > y_e$ and $y_b > y_e$ then 2 endif
отсечката е частично видима или пресича продължението на диагоналите,
оставайки невидима.
Определя се пресечната точка с прозореца
2 *отсечката е невидима*
3 *преход към следваща отсечка.*

Приведените тестове за пълна видимост или невидимост може да се формализират чрез метода на Коен и Съзерланд. В този метод за определяне на тази от деветте области, на която принадлежи края на реброто, се извежда четириразреден битов код. Кодовете на тези области са показани на фиг. 10.1.2. Крайният десен бит на кода се счита за първи. В съответния бит се поставя 1 при следните условия (фиг. 10.1.2):

за първи бит - ако точката е в ляво от прозореца
за втори бит - ако точката е в дясно от прозореца
за трети бит - ако точката е отдолу на прозореца
за четвърти бит - ако точката е отгоре на прозореца
В противен случай в бита се внася 0.



Фиг. 10.1.2

Ако кодовете на двата края на реброто са равни на 0, то и двете точки лежат вътре в прозореца и отсечката е видима. Кодовете на крайните точки могат да се използват също за отхвърляне на напълно невидимите отсечки.

Ако побитовото логическо произведение на кодовете на крайните точки на отсечка **не е равно на нула**, отсечката е **напълно невидима**. Ако това произведение **е равно на нула**, отсечката може да е **изцяло или частично видима** или даже **изцяло невидима**. **Затова за определяне на пълната видимост е необходимо да се проверяват стойностите на кодовете на двата края на отсечката поотделно.**

Ако са намерени изцяло видимите и невидимите отсечки, то на подпрограмата, изчисляваща пресичането на отсечките се предават само отсечките, които вероятно са частично видими (за тях резултатът от логическото умножение на кодовете на крайните им точки е 0). Тази подпрограма трябва също така правилно да определя и предадените и такива изцяло невидими отсечки.

Пресичането на две отсечки може да се търси по параметрични и непараметрични начини. Уравнението на безкрайна права, преминаваща през точки $P_1(x_1, y_1)$ и $P_2(x_2, y_2)$ има вида:

$$y = m(x - x_1) + y_1 \text{ или } y = m(x - x_1) + y_2,$$

където $m = (y_2 - y_1) / (x_2 - x_1)$ е наклона на дадената права. Точките на пресичането на тази права със страните на прозореца имат следните координати:

с лявата: $x_l, y = m(x_l - x_1) + y_1$ $m \neq \infty$

с дясната: $x_d, y = m(x_d - x_1) + y_1$ $m \neq \infty$

с горната: $y_e, x = x_1 + (1/m)(y_e - y_1)$ $m \neq 0$

с долната: $y_d, x = x_1 + (1/m)(y_d - y_1)$ $m \neq 0$

За да бъде ефективен даденият алгоритъм, трябва да се разгледат и някои частни случаи. Ако наклонът е безкраен, то отсечката е успоредна на лявата и дясната страна на прозореца и се търси пресичането само с горната и долна страна. Аналогично, ако наклонът е 0, отсечката е успоредна на горната и долна страна на прозореца и се търси пресичане

само с лявата и дясна страна. Ако кодът на един от краищата на отсечката е равен на 0, то този край лежи вътре в прозореца и отсечката пресича само едната страна на прозореца.

Алгоритъм за двумерно отсичане [11]

P_1, P_2 - крайни точки на отсечката

P_1', P_2' - крайни точки на видимата част на отсечката

x_l, x_d, y_d, y_z - координати на лявата, дясната, долната и горната страна на прозореца

Flag-признак за видимост, равен на 0 при видимост, -1 при невидимост

Изчисляват се кодовете на крайните точки.

Занасят се тези кодове в два масива T1kod и T2kod с размерност 1 X4 всеки.

За първия край P_1 :

if $x_1 < x_l$ then T1kod(4)=1 else T1kod(4)=0 endif

if $x_1 > x_d$ then T1kod(3)=1 else T1kod(3)=0 endif

if $y_1 < y_d$ then T1kod(2)=1 else T1kod(2)=0 endif

if $y_1 > y_z$ then T1kod(1)=1 else T1kod(1)=0 endif

за втория край P_2

if $x_2 < x_l$ then T2kod(4)=1 else T2kod(4)=0 endif

if $x_2 > x_d$ then T2kod(3)=1 else T2kod(3)=0 endif

if $y_2 < y_d$ then T2kod(2)=1 else T2kod(2)=0 endif

if $y_2 > y_z$ then T2kod(1)=1 else T2kod(1)=0 endif

Инициализация на признака за видимост и видими крайни точки.

Инициализация на m - много голямо число, имитиращо безкраен наклон.

Flag=0

$P_1' = P_1$

$P_2' = P_2$

m=голямо число

Проверка за пълна видимост на отсечката.

Suma1=0

Suma2=0

for i=1 to 4

Suma1=Suma1+T1kod(i)

Suma2=Suma2+T2kod(i)

next i

if Suma1=0 and Suma2=0 then 7 endif

Отсечката не е напълно видима.

Проверка на случая за тривиална невидимост.

Изчисляване на логическото произведение (proizw) на кодовете на крайните точки на отсечката.

proizw=0

for i= 1 to 4

proizw=proizw + Цялата част ((T1kod(i)+T2kod(i))/2)

if proizw<>0 then

Flag=-1

goto 7

endif

next i

Отсечката може да бъде частично видима.

Проверка за попадане на първата точка вътре в прозореца.

if Suma1=0 then

nomer=1

$P_1' = P_1$

$P = P_2$

go to 2

endif

Проверка за попадане на втората точка вътре в прозореца.

if Suma2=0 then

```

    nomer=2
     $P_1' = P_2$ 
     $P = P_1$ 
    goto 2
endif
Вътре в прозореца няма краища на отсечката.
Инициализация на номера на края на отсечката.
nomer=0
1 if nomer=<>0 then  $P'_{nomer} = P$  endif
  nomer=nomer+1
  if nomer>2 then 7 endif
   $P = P_{nomer}$ 
Проверка за пресичане с левия край.
Проверка за вертикалност на отсечката.
2 if  $(x_2 - x_1) = 0$  then 4 endif
   $m = (y_2 - y_1) / (x_2 - x_1)$ 
  if  $x_l < P_x$  then 3 endif
   $y = m * (x_l - P_x) + P_y$ 
  if  $y > y_e$  then 3 endif
  if  $y < y_d$  then 3 endif
Намерено е коректно пресичане.
   $P_y = y$ 
   $P_x = x_l$ 
  goto 1
Проверка за пресичане с десния край.
3 if  $x_d > P_x$  then 4 endif
   $y = m * (x_d - P_x) + P_y$ 
  if  $y > y_e$  then 4 endif
  if  $y < y_d$  then 4 endif
Намерено е коректно пресичане.
   $P_y = y$ 
   $P_x = x_d$ 
  goto 1
Проверка за пресичане с горния край.
Проверка за хоризонталност на отсечката.
4 if  $m=0$  then 1 endif
  if  $y_e > P_y$  then 5 endif
   $x = (1 / m) * (y_e - P_y) + P_x$ 
  if  $x < x_l$  then 5 endif
  if  $x > x_d$  then 5 endif
Намерено е коректно пресичане.
   $P_x = x$ 
   $P_y = y_e$ 
  goto 1
Проверка за пресичане с долния край.
5 if  $y_d < P_y$  then грешка endif
   $x = (1 / m) * (y_d - P_y) + P_x$ 
  if  $x < x_l$  then 6 endif
  if  $x > x_d$  then 6 endif
Намерено е коректно пресичане.
   $P_x = x$ 
   $P_y = y_d$ 
  goto 1
Отсечката е невидима.
6 Flag= -1
Завършване на работата и извикване на процедура за чертане.
7 if Flag= -1 then 8

```

Draw (P_1', P_2')
Преминаване към обработка на следващата отсечка.
8 finish

10.1.3. Поле на изхода

След създаване на прозорец на екрана е възможно да се изпълнят операциите за преобразуване над отделената част от изображението, а след това резултатът да се пренесе в зададена правоъгълна област от екрана, **наречена поле на графичното представяне на обектите или поле на изхода или още видово поле (Viewport)**. Размерите на полето на изхода може да се различават от размерите на прозореца. Програмата, преобразуваща областта на прозореца в поле на изхода, трябва да преобразува координатите на всички точки в прозореца.

Новите координати на точките в полето на изхода са свързани с координатите на съответната точка в прозореца със следните зависимости:

$$x_n = (x - x_w) \cdot (w_v / w_w) + x_v$$

$$y_n = (y - y_w) \cdot (h_v / h_w) + y_v$$

където w_v / w_w и h_v / h_w са коефициентите на мащабиране.

Ако двата коефициента не са равни, се получава изкривяване на изображението. Членовете x_v и y_v определят операцията преместване.

10.2. ГЕНЕРАЦИЯ НА ЗАПЪЛНЕНИ ОБЛАСТИ

Една от уникалните възможности на растровото графично устройство е възможността да се представят **‘плътни’**, в определен смисъл, области. Обикновено за целта се използват **методи за запълване на контури**, делящи се на две широки категории: **растрова развивка** и **верижно запълване**.

В първия метод се определя по време на сканиране на реда лежи ли точката вътре в контура. Методът е приложим и за векторните дисплеи, където се използва за щриховка или оцветяване на контури.

В методите на верижното запълване се предполага, че е известна някоя точка вътре в затворения контур. В тези алгоритми се търсят **точки, съседни с верижната** и разположени **вътре** в контура. Ако съседна точка не е разположена вътре, значи, че **е намерена граница на контура**. Ако точката е **вътре в контура, тя става верижна** и търсенето продължава рекурсивно. Подобни алгоритми са приложими само към растрови устройства.

Много **затворени контури** се явяват **прости многоъгълници**. Ако контурът се състои от криви линии, той може да се апроксимира с подходящ многоъгълник или многоъгълници. Най-лесният метод за запълване на многоъгълник се състои в **проверка за принадлежност към вътрешността на многоъгълника във всеки пиксел** от растера. Този метод, обаче, е твърде разточителен, тъй като болшинството пиксели лежат извън многоъгълника. Загубите могат да се намалят, като се намери най-малкия правоъгълник, съдържащ в себе си многоъгълника. Тогава се проверяват само вътрешните точки на тази обвивка.

10.2.1. Растрова развивка на многоъгълници

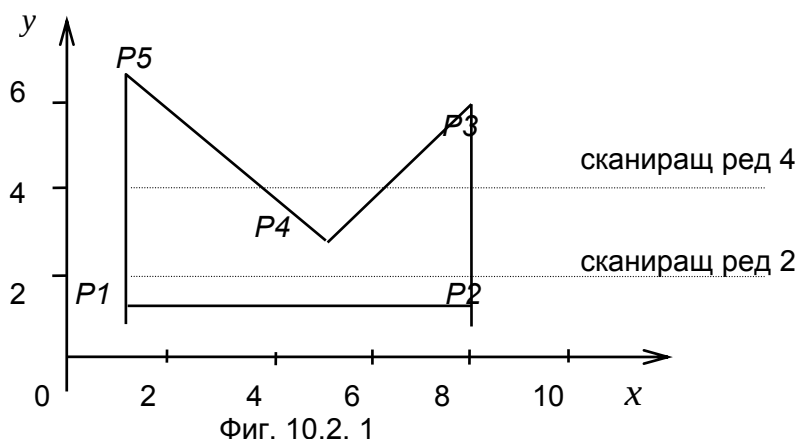
Може да се използва факта, че съседните пиксели вероятно имат еднакви характеристики (с изключение на пикселите на граничните ребра). Това свойство се нарича пространствена кохерентност. Характеристиките на пикселите на даден сканиращ ред се изменят само там, където реброто на многоъгълника пресича реда. Тези пресичания делят сканиращия ред на области.

За многоъгълника на фиг. 10.2.1 ред 2 пресича последния при $x=1$ и $x=8$. Получават се три области:

$x < 1$ - извън многоъгълника

$1 \leq x \leq 8$ - вътре в многоъгълника

$x > 8$ - извън многоъгълника



Ред 4 се дели на 5 области:

$x < 1$ - извън многоъгълника

$1 \leq x \leq 4$ - вътре в многоъгълника

$4 < x < 6$ - извън многоъгълника

$6 \leq x \leq 8$ - вътре в многоъгълника

$x > 8$ - извън многоъгълника

При определяне на интензивността, цвета и оттенъка на пикселите на сканирания ред се разглеждат двойки - сортирани точки на пресичане. За всеки интервал, задаван от двойка пресичания се използва интензивността или цвета на запълвания многоъгълник. За интервалите между двойките пресичания и крайните (от началото на сканирания ред до първата точка на пресичане и от последната точка на пресичане до края на сканирания ред) се използва фоновата интензивност или цвят. На фиг. 10.2.1 за реда 4 с цвят на фона на картината ще бъдат изобразени пикселите от 0 до 1, от 4 до 6, от 8 до 10, докато пикселите от 1 до 4 и от 6 до 8 ще бъдат изобразени с цвета на запълнения многоъгълник. На тази основа могат да бъдат синтезирани различни алгоритми за запълване на области. По-долу се разглеждат само някои от тях.

Алгоритъм с подреден списък на ребрата [11]

Подготовка на данните:

Определят се за всяко ребро на многоъгълника точките на пресичане със сканиращите редове, проведени през средата на интервалите. За това може да се използва алгоритъма на Брезенхайм или на ЦДА. Хоризонталните ребра се игнорират. Внася се всяко пресичане $(x, y+1/2)$ в списък. Сортира се списъкът по редове и по нарастване на x в реда; т.е. (x_1, y_1) предшества (x_2, y_2) , ако $y_1 > y_2$ или $y_1 = y_2$ и $x_1 \leq x_2$.

Преобразуване на данните в растрова форма:

Отделят се от сортирания списък двойките елементи (x_1, y_1) и (x_2, y_2) . Структурата на списъка гарантира, че $y = y_1 = y_2$ и $x_1 \leq x_2$. Активират се на сканирания ред y пиксели за целите стойности на x , такива, че $x_1 \leq x+1/2 \leq x_2$.

Алгоритъм за верижно запълване [11]

Тук се предполага, че е **известен поне един пиксел от вътрешната област на многоъгълника**. Алгоритъмът се опитва да намери и оцвети всички останали пиксели, принадлежащи на областта. Областите могат да бъдат или вътрешни, или гранично-определени. Ако областта е вътрешно-определена, всички пиксели, принадлежащи към вътрешната част имат един цвят или интензитет, а пикселите, външни на областта, имат друг цвят.

Ако областта е гранично-определена, всички пиксели на границата имат определена стойност или цвят. Пикселите от вътрешните части не могат да имат тази стойност. Вътрешните или гранично-определени области могат да бъдат 4 или 8-свързани. Ако областта е 4-свързана, то всеки пиксел в областта може да се достигне с комбинация от движения в четири направления- наляво, надясно, нагоре, надолу. За 8-свързаните области

пиксела може да се достигне с комбинация от движения в 2 хоризонтални, 2 вертикални и 4 диагонални направления.

Алгоритъм за запълване на гранично-определена област, използващ стек. [11]

Помества се верижният пиксел в стека.

Докато стека не е празен

Извлича се пиксел от стека.

Присвоява се на пиксела необходимата стойност.

За всеки от съседните към текущия 4-свързани пиксели се проверява явява ли се той граничен пиксел или не е ли вече присвоено необходимото. В тези два случая пикселът се игнорира. В противен случай пикселът се помества в стека.

Алгоритъмът може да се модифицира за 8-свързани области, ако се разглеждат 8-свързани пиксели, а не само 4-свързани.

Алгоритъм за запълване [11]

Точката (x, y) определя верижния пиксел

Push - процедура, която помества пиксела в стека

Pop- процедура, която извлича пиксел от стека

Пиксел(x, y) = Точка(x, y)

инициализира се стека

Push Пиксел(x, y)

while (стека не е празен)

извлича се пиксел от стека

Pop Пиксел(x, y)

if Пиксел(x, y) <> Нова_стойност then

Пиксел(x, y) = Нова_стойност

endif

проверяват се съседните пиксели дали трябва да се поместят в стека

if Пиксел(x+1, y) <> Нова_стойност . and. Пиксел(x+1, y) <> Гран_стойност then

Push Пиксел(x+1, y) endif

if Пиксел(x, y+1) <> Нова_стойност . and. Пиксел(x, y+1) <> Гран_стойност then

Push Пиксел(x, y+1) endif

if Пиксел(x-1, y) <> Нова_стойност . and. Пиксел(x-1, y) <> Гран_стойност then

Push Пиксел(x-1, y) endif

if Пиксел(x, y-1) <> Нова_стойност . and. Пиксел(x, y-1) <> Гран_стойност then

Push Пиксел(x, y-1) endif

end while

finish

В алгоритма се проверяват и поместват в стека 4-свързани пиксели, започвайки с десния на текущия пиксел. Направлението при обхождането на пикселите е обратно на часовата стрелка.

10.3 РАЗРЯЗВАНЕ НА МНОГОЪГЪЛНИЦИ

Терминът “разрязване” се използва за обозначаване на процеса на установяване пресича ли се права (или многоъгълник) с даден многоъгълник. Целта на прилагането на процедурата на разрязване е да се определи попада ли някаква съвкупност от обекти в определен прозорец на екрана.

Разрязването се използва за решаване на задачата за видимост (за да се определи дали един обект засенчва друг, предварително трябва да се установи дали те се пресичат). Друго приложение е разбиването на неизпъкнал многоъгълник на изпъкнали подмногоъгълници, което се използва в задачите на компютърната графика и разпознаването на образи.

10.3.1. Разрязване на произволна отсечка от права с произволен изпъкнал многоъгълник

Нека $A_1, A_2, \dots, A_n$ са върховете на изпъкнал плосък многоъгълник Π . Необходимо е да се определи каква част от отсечка на правата L (зададена с крайните си точки P_1 и P_2) лежи вътре в този многоъгълник.

1. Изчисляват се n стойности на величината

$$S_i = \det(\mathbf{A}_i, \mathbf{P}_1, \mathbf{P}_2) = X_i(Y_1W_2 - W_1Y_2) + Y_i(W_1X_2 - X_1W_2) + W_i(X_1Y_2 - Y_1X_2), \quad i = 1, 2, \dots, n \quad (10.3.1)$$

Където $\mathbf{A}_i$ – вектор с координатите (X_i, Y_i) на върховете A_i на многоъгълника и $\mathbf{P}_j$ ($j=1,2$) – вектор на координатите (x_j, y_j) на крайните точки на отсечката P_j . Получените резултати могат да съответстват на един от следните случаи:

1. Всички стойности на S_i са ненулеви и имат еднакъв знак. Тогава всички върхове на многоъгълника са разположени от едната страна на отсечката от правата L и никаква част от отсечката не лежи вътре в многоъгълника.
2. Едно от стойностите на S_i е равно на 0, а всички останали стойности имат еднакъв знак. Тогава правата преминава през един от върховете на многоъгълника, а всички останали върхове са разположени от едната страна на правата.
3. Две стойности на S_i са равни на 0, а всички останали имат еднакъв знак. Тъй като многоъгълникът е изпъкнал, това означава, че два съседни върха на многоъгълника лежат върху отсечката от правата L , т.е. една от страните на многоъгълника е разположена на същата права, на която лежи и отсечката от правата L .
4. Едно или две стойности на S_i са равни на нула, а останалите стойности са с различни знаци. Тогава отсечката от правата L пресича многоъгълника Π , преминавайки през един или два негови върха.
5. Всички S_i имат ненулеви стойности и знаците им са различни. Тогава при обхождане на многоъгълника по периметъра му знака се изменя два пъти. За означаване на двойката върхове, в които настъпва промяна на знака ще се използват символите A_j, A_{j+1} и A_k, A_{k+1} . В случай 4 S_i може да приема ненулево значение в един от върховете на едната или двете двойки. Отсечката от правата, съединяваща върховете на първата двойка се означава с L_1 , а отсечката, съединяваща върховете на другата двойка – с L_2 .

След установяване на пресичането на правата, съдържаща отсечката L с разглеждания многоъгълник (случаи 4 и 5), е необходимо да се определи разположението на точките P_1 и P_2 относно отсечките от правите L_1 и L_2 , тоест знаците на следните 4 величини:

$$U_1 = \det(\mathbf{P}_1, \mathbf{A}_j, \mathbf{A}_{j+1}) = x_1(Y_jW_{j+1} - W_jY_{j+1}) + y_1(W_jX_{j+1} - X_jW_{j+1}) + w_1(X_jY_{j+1} - Y_jX_{j+1}) \quad (10.3.2 \text{ a})$$

(разположението на точка P_1 относно отсечката от правата L_1);

$$U_2 = \det(\mathbf{P}_2, \mathbf{A}_j, \mathbf{A}_{j+1}) = x_2(Y_jW_{j+1} - W_jY_{j+1}) + y_2(W_jX_{j+1} - X_jW_{j+1}) + w_2(X_jY_{j+1} - Y_jX_{j+1}) \quad (10.3.2 \text{ b})$$

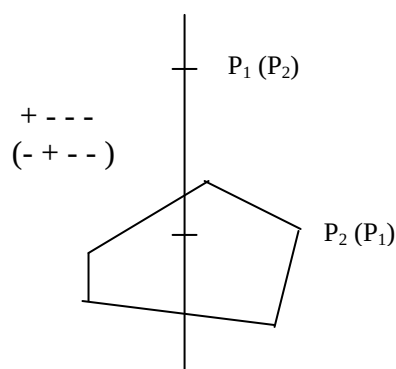
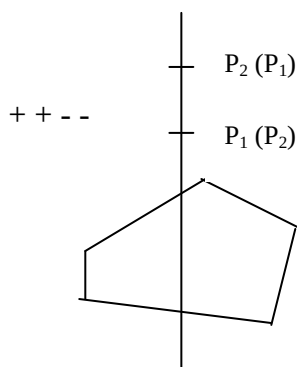
(разположението на точка P_2 относно отсечката от правата L_1);

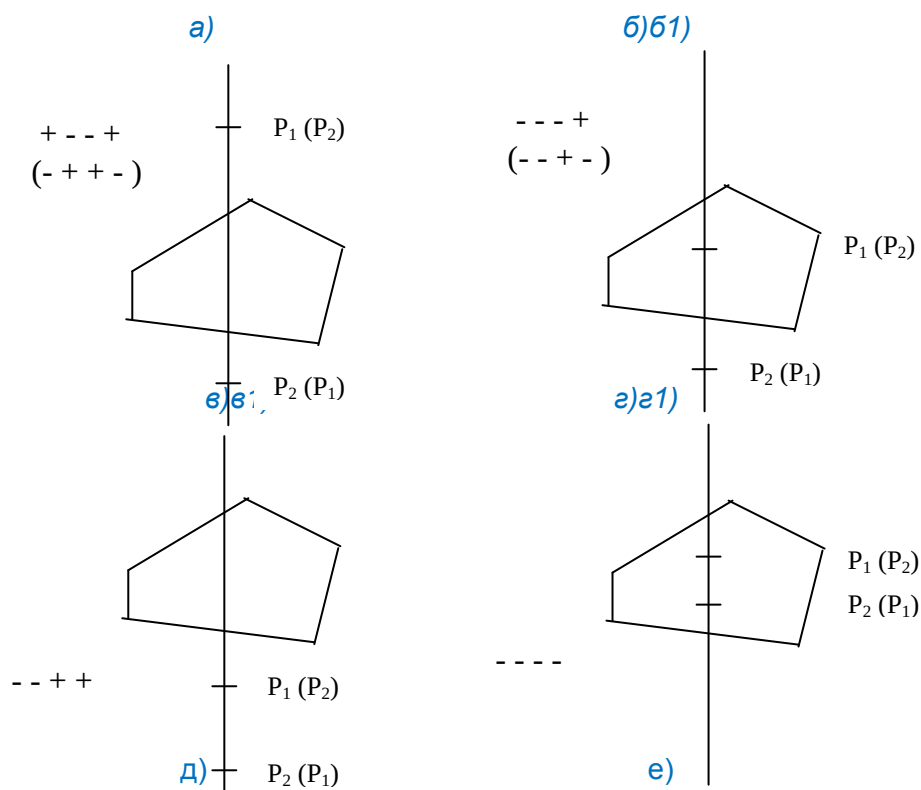
$$U_3 = \det(\mathbf{P}_1, \mathbf{A}_k, \mathbf{A}_{k+1}) = x_1(Y_kW_{k+1} - W_kY_{k+1}) + y_1(W_kX_{k+1} - X_kW_{k+1}) + w_1(X_kY_{k+1} - Y_kX_{k+1}) \quad (10.3.2 \text{ в})$$

(разположението на точка P_1 относно отсечката от правата L_2);

$$U_4 = \det(\mathbf{P}_2, \mathbf{A}_k, \mathbf{A}_{k+1}) = x_2(Y_kW_{k+1} - W_kY_{k+1}) + y_2(W_kX_{k+1} - X_kW_{k+1}) + w_2(X_kY_{k+1} - Y_kX_{k+1}) \quad (10.3.2 \text{ г})$$

(разположението на точка P_2 относно отсечката от правата L_2);





Фиг.10.3.1. Шест възможни варианта на разрязване на произволна отсечка от права с произволен многоъгълник

При подреждане на ориентацията на страните на многоъгълника по часовата стрелка величината U_i има в уравнения (2) отрицателен знак, ако ако точката лежи вътре в многоъгълника. Значението $U_i = 0$ ако една от точките P_1 и P_2 лежи на една от страните на многоъгълника.

На фиг. 10.3.1 са показани различни варианти на разположение на точките P_1 и P_2 относно две прави, а таблица 10.3.1 съдържа знаците на U_i и резултата от определянето на крайните точки на възпроизвежданата отсечка.

Обозначения: Q_1 и Q_2 – крайните точки на отсечката от правата, чието изображение трябва да се възпроизведе; $L \cap L_i$ ($i=1,2$) – точка на пресичане на отсечките L и L_i .

Фиг. 10.3.1	U_1	U_2	U_3	U_4	Q_1	Q_2
а)	+	+	-	-	невидима	невидима
б)	+	-	-	-	$L \cap L_1$	P_2
б1)	-	+	-	-	P_1	$L \cap L_1$
в)	+	-	-	+	$L \cap L_1$	$L \cap L_2$
в1)	-	+	+	-	$L \cap L_2$	$L \cap L_1$
г)	-	-	-	+	P_1	$L \cap L_2$
г1)	-	-	+	-	$L \cap L_2$	P_2
д)	-	-	+	+	невидима	невидима
е)	-	-	-	-	P_1	P_2

Таблица 10.3.1. Разположение на произволна отсечка от права относно произволен многоъгълник

Алгоритъм за разрязване на произволна отсечка от права с произволен изпъкнал многоъгълник

Означения. Масив $A(n \times 3)$ съдържа тройки стойности на еднородни координати n на върховете на многоъгълника. Масив $P(2 \times 2)$ съдържа координатите на крайните точки на отсечката от правата L . Масив $B(2 \times 3)$ съхранява първата от двойките върхове, разположени

от различни страни на правата, съдържаща отсечката L . Масив $C(2 \times 3)$ съдържа втория връх на тези двойки. S_i и U_i се определят от (10.3.1) и (10.3.2).

Алгоритъмът определя Q_1 и Q_2 – крайни точки на отсечката, разположена вътре в многоъгълника.

1. $K=0$ (K - брояч)
2. $v=S_i$ (S_i се определя от (1) при използване на $A(1,1)$ в качеството на X_1 , $A(1,2)$ – в качеството на Y_1 и т.н.
3. **For** $i=2$ до n **do**:

Begin
 4. на променливата u се присвоява значение, съответстващо на знака на S_i , определен от (1) при използване на $A(i,1)$ в качеството на X_i , $A(i,2)$ – в качеството на Y_i и т.н.
 5. **If** v различно от u **then** $K=K+1$; $B(K,i)=A(i-1,i)$; $C(K,i)=A(i,i)$; $v=u$
 6. **If** $K=2$ **then break**.
 - End.**
 7. **If** $K=0$ **then exit**.
 8. **Else do**:

Begin
 9. Определя се уравнение (2); от масив $B(1,*)$ се взема точка A_j , от масива $C(1,*)$ – A_{j+1} ; от масива $B(2,*)$ – A_k и от $C(2,*)$ – A_{k+1} . Изчисляват се U_1, U_2, U_3, U_4 .
 10. **If** U_1 и U_3 имат еднакви знаци противоположни на U_2 и U_4 **then exit**.
 11. **Else do**:

Begin
 12. **For** $m=1$ до 2 **do**:

Begin
 13. **If** U_m и U_{m+2} имат противоположни знаци **then do**:

Begin
 14. **If** U_m има отрицателно значение, Q_m се квалифицира като точка на пресичане на отсечките от правите L и L_2 .
 15. **Else** Q_m се квалифицира като точка на пресичане на отсечките от правите L и L_1 .
 - End.**
 16. **Else** в качеството на Q_m се използва точка P_m .
 - End.**
 - End.**
 17. Възпроизвежда се отсечката от правата, съединяваща точките Q_1 и Q_2 .
 18. **Край на алгоритъма.**

10.4 OPENGL – ЗАПЪЛВАНЕ НА ЗАТВОРЕНИ ОБЛАСТИ С РАЗЛИЧНИ ТЕКСТУРИ

Поставянето на текстура върху обект е запълване на контурите с шриховка с определени качества. За целта се използва и **Mipmapping** технологията, която е сравнително нова и игрите през последните 5 години използват точно нея.

Използването на текстура, не пречи да се използва динамична светлина. След текстурата се определят светлинния източник и материала на обекта. Изображенията с текстури могат да се заредят най-лесно от *.bmp файл. След като се зареди изображението, трябва да бъде построена текстура и след това съхранена в масив, откъдето може да се използва за различни обекти.

Първото се разрешава 2D текстурирането **glEnable(GL_TEXTURE_2D)** в **main()** функция или друга подобна, която се извиква преди рендериращата функция.

След това е нужно да се определи по-какъв начин ще се наслаждава текстурата върху обекта. Това става с функцията :

void glTexEnv{if}(GLenum target, GLenum pname, GLint param) - тя приема три аргумента. Първият трябва да е **GL_TEXTURE_ENV**, вторият аргумент - **GL_TEXTURE_ENV_MODE**, а последният може да бъде една от константите :

GL_MODULATE - цвета на пикселите от текстурата се умножава по цвета на обекта.

GL_DECAL - цвета на пикселите от текстурата заменя цвета на обекта.

GL_BLEND - цвета на текстурата се смесва с предварително определен цвят-константа и след това с цвета на обекта. За да се определи константния цвят трябва да се извика функцията :

void glTexEnvfv(GLenum target, GLenum pname, const GLint *params) - тя приема 3 аргумента. Първия трябва да е отново **GL_TEXTURE_ENV**, но за втори аргумент вече трябва да се подадете **GL_TEXTURE_ENV_COLOR**, а третия аргумент е съответно **RGBA** масива с цвета.

Когато се използва динамична светлина, тя се отразява на цвета на обекта. Например ако цвета на обекта е бял, при използването на динамичната светлина по-неосветените части стават по-тъмни и дори сиви. Ако се иска динамичната светлина да се отразява на повърхността на текстурата трябва да се избере опцията **GL_MODULATE**. Така всеки път когато цвета на пиксела на текстурата се умножи по този на обекта (при включена светлина цвета на обекта прелива в зависимост от количеството на осветяване) се получава илюзията, че текстурата рефлектира на светлината, а не цвета на обекта. Има текстура, която се променя в зависимост от идващата светлина.

Следва да се зареди изображението на текстурата, т.е. да се конвертира в масив от **RGB** стойности. В библиотеката **GLAUX** съществува функция, която конвертира автоматично ***.bmp** файл с прототип :

AUX_RGBImageRec* auxDIBImageLoad("име.bmp") - както се вижда тя приема името на **bmp** файла и връща указател към структурна променлива от тип **AUX_RGBImageRec**. Структурата **AUX_RGBImageRec** е дефинирана в библиотеката **GLAUX** и съдържа няколко члена. Това са **sizeX** , **sizeY** от тип **int**, в които се запазват съответно ширината и дължината на зареденото изображение и ***data** от тип **unsigned char**, където се запазват **RGB** стойностите на изображението.

Пример за зареждане на изображението „wood.bmp” :

AUX_RGBImageRec *image;

Пример: **image = auxDIBImageLoad("wood.bmp");**

Следващата стъпка е генерацията на "име" за всяка една от текстурите. Това става с функцията :

void glGenTextures(GLsizei n, GLuint *textures) - тя приема два аргумента. Първият определя броя на "имената" на текстурите, които да бъдат генерирани, а втория аргумент трябва да бъде адреса на масив от тип **unsigned int**, в който те да се съдържат. Трябва да се даде адреса на първата клетка от масив **texture[3]** т.е. **&texture[0]**. Имената на текстурите не са задължителни, но ако няма такива, ще може да се използва оптимално само по една текстура.

Пример на създаване на имена за три текстури:

glGenTextures(3, &textures[0]) , като предварително са дефинирани в масива **GLuint textures[3]**.

Построяването на текстура/и. предполага използването на функцията за назначаване на текуща текстура, използвайки името и, с генерираните вече имена от масива, който е посочен с функцията **glGenTextures()**. Тази функция е:

void glBindTexture(GLenum target, GLuint texture) - тя приема 2 аргумента. Първият трябва да е константата (двумерна или тримерна) като **GL_TEXTURE_2D**, определяща, че съответното име се отнася за **2D** текстура, а втория – поредния номер на клетката с желаното име от масива с генерираните вече имена.

Препоръчително е първо да се построят/зададат нужните за програмата текстури и после да се избира, коя да бъде използвана за конкретните графични обекти.

Пример: **glBindTexture(GL_TEXTURE_2D, texture[0]);**

След като са свързани първата текстура с първия инд. номер от масива идва ред тя да бъде построена. За построяването на дадена текстура са възможни две функции.

Първата от тях е стандартната функция в **OpenGL** - **glTexImage2D()** за построяване на **2D** текстури, а втория вариант е използването на технологията **Mipmapping** с функцията **gluBuild2DMipmaps()** от библиотеката **GLU**. Ако се използва **glTexImage2D()** за

построяване на дадена текстура, размерите на изображение трябва да са степен на 2. Примерно: 64x128 или 256x256 и т.н.

Прототип на функцията **glTexImage2D()** :

void glTexImage2D(GLenum target, GLint level, GLint internalformat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, const GLvoid *pixels);

Функцията приема 9 аргумента. Първият аргумент трябва да е константата **GL_TEXTURE_2D**, вторият аргумент определя нивото на детайлите на текстурата и по подразбиране е 0, което значи че текстурата си остава каквато е т.е. базово ниво. Третия аргумент определя броя на цветовете компоненти, които се използват за всеки пиксел. В **Color-Index** режим те са 1, в **RGB** - 3, а в **RGBA** съответно - 4. Най-често цветовете компоненти са 3. Четвъртия и петия аргумент определят ширината и дължината на текстурата, които се съдържат съответно в **image->sizeX** и **image->sizeY**, където ***image** е указател към структурна променлива от тип **AUX_RGBImageRec**. Шестият аргумент определя ширината на рамката на всяка текстура върху даден обект т.е. пикселите които се изпускат при нанасянето на следваща текстура, (за предпочитане стойност 0). Седмият аргумент указва типа на цветовете, които се използват и по принцип е една от константите **GL_RGB**, **GL_RGBA** или **GL_COLOR_INDEX**. В случая - **GL_RGB**. Осмият аргумент указва типа на данните в масива с **RGB** стойностите. При използването на библиотеката **GLAUX** за типа на данните е нужно да подадете аргумента **GL_UNSIGNED_BYTE**. Последния девети аргумент представлява масива със **RGB** стойностите за всеки пиксел от нашето изображение т.е. подавате като аргумент: **image->data**.

Пример:

glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, image->sizeX, image->sizeY, 0, GL_RGB, GL_UNSIGNED_BYTE, image->data);

Вторият начин за построяване на текстура с функцията **gluBuild2DMipmaps()**. **Mipmap** текстурите са много по-добри от стандартните създадени с **glTexImage2D**. Тяхното предимство се състои в това, че за разлика от обикновените текстурите, които биват "скалирани" за да се нагодят за съответната повърхност, при **Mipmapping** - а те се построяват автоматично поредица от текстурите с различна големина и качество, като **OpenGL** избира най - подходящата текстура за дадения момент. Например ако построите обикновена текстура от високо-качествено изображение, при малко разстояние от камерата до текстурирания обект, всичко ще е нормално, но ако се отдалечи камерата текстурата ще бъде "скалирана" към по-малки размери, ще започне да трепти, което се дължи именно на "скалирането", **OpenGL** ще продължи да я рендерира със всички детайли, независимо от това, че вече не се забелязват, поради малките им размери. Ако се използва **Mipmapping**, това няма да се случи, защото ще бъде построена нова текстура отговаряща на новите по-малки размери на обекта, няма да има трептене, както и малките детайли, които вече и без това не се забелязват, няма да се съдържат в нея. Новата текстура ще бъде с по-ниско качество, но ще изглежда отново задоволително, понеже камерата е по - надалеч от нея. Ако пък се приближи близко към обекта ще бъде показана отново текстурата с най-високо качество. При **Mipmapping** текстурите могат да се ползват изображения от всякакъв размер.

Прототип на функцията **gluBuild2DMipmaps()** :

int gluBuild2DMipmaps(GLenum target, GLint components, GLint width, GLint height, GLenum format, GLenum type, const void *data);

Тя приема 7 аргумента и връща 0, ако няма проблеми при построяването на **Mipmap** текстурата (не е задължително да се проверяват какво връща функцията). Първият аргумент може да е константата **GL_TEXTURE_2D**. Вторият указва броя на цветовете компоненти, обикновено 3. Четвъртия и петия аргумент определят ширината и дължината на текстурата. Петият - константата показваща типа на цветовете, които се използват - в случая това е **GL_RGB**, последните два аргумента са типа на данните в масива и самия масив. Аргументите са аналогични с тези от функцията **glTexImage2D()**. Функцията заедно с подадените аргументи:

gluBuild2DMipmaps(GL_TEXTURE_2D, 3, image->sizeX, image->sizeY, GL_RGB, GL_UNSIGNED_BYTE, image->data);

След построяването на текстура, независимо с коя от двете функции, трябва да се избере начин на филтрация на текстурата, от което зависи и качеството на текстурата при визуализация. Това става с функцията:

void glTexParameter{if}(GLenum target, GLenum pname, GLint param) - приема 3 аргумента. Първият трябва може да е **GL_TEXTURE_2D**, съответно за **2D** текстури. Вторият аргумент определя филтъра и може да е един от аргументите :

GL_TEXTURE_MAG_FILTER - филтрация, когато текстурата се увеличава на по-голяма площ от стандартните ѝ размери т.е. качество на текстурата при рендериране на малко разстояние до камерата.

GL_TEXTURE_MIN_FILTER - филтрация, когато текстурата се намалява от стандартните ѝ размери т.е. при големи разстояния от текстурата.

Третият аргумент определя качеството на определената филтрация :

При **MAG** филтъра може да се избере един от аргументите: **GL_NEAREST** или **GL_LINEAR**, съответно за ниско или висококачествена филтрация. Висококачествената филтрация има и своите недостатъци - тя усложнява допълнително работата на графичния ускорител и от това страда общата производителност.

При **MIN** филтъра нещата са по-сложни. Ако при построяване на текстурата е използвана функцията **glTexImage2D()** то избора се свежда отново до аргументите **GL_NEAREST** и **GL_LINEAR**, но при използването на **gluBuild2DMipmaps()** вече се предоставят четири нови аргумента и това са:

GL_NEAREST_MIPMAP_NEAREST

GL_NEAREST_MIPMAP_LINEAR

GL_LINEAR_MIPMAP_NEAREST

GL_LINEAR_MIPMAP_LINEAR

Крайната част от всеки аргумент определя качеството на самата замяна на **Mipmap** текстурите (или по-точно тя бива по-плавна при избирането на филтрация завършваща на **LINEAR**). Режимът, който постига най-добро съотношение между качество/производителност е **GL_LINEAR_MIPMAP_NEAREST**, като най-качественият режим разбира се е **GL_LINEAR_MIPMAP_LINEAR**.

В графичните опции на някои игри има избор между **BILINEAR** и **TRILINEAR** филтрация за текстурите. Това са по-кратките имена на режимите **GL_LINEAR_MIPMAP_NEAREST** и **GL_LINEAR_MIPMAP_LINEAR**.

Може да се използва **GL_LINEAR** и **GL_NEAREST**, но вече текстура няма да е **Mipmap**.

Пример за една **Mipmap** текстура:

```
glTexParameteri( GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR );
glTexParameteri( GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
GL_LINEAR_MIPMAP_NEAREST );
```

След построяването на нужните текстури, избиране на режим на филтриране, следва запълването (разпростирането) с текстурата на затворената област. За целта освен определяне на координатите на върховете на рисувания обект трябва да се определят и координати за текстурата. Координатите на текстурата определят коя точка от текстурата се намира върху съответния връх. Важно е да се отбележи, че координатите се определят само веднъж и нататък **OpenGL** се грижи сам за своята работа, при всякакви завъртания или движения на обекта. Определянето на координатите на текстурата става с функцията: **glTexCoord*()**, която има доста разновидности, но най-използваната е **glTexCoord2f()**. Тя приема 2 стойности съответно за x и y, указващи местоположението на съответната точка от текстурата. Например, ако има квадрат, върху който да се сложи текстура и се започне определянето на върховете му обратно на часовниковата стрелка от долния ляв ъгъл, то координатите на текстурата ще бъдат съответно : (0, 0) - първи пиксел от текстурата долу в ляво, (1, 0) - последен пиксел от текстурата долу в дясно, (1, 1) - последен пиксел от текстурата горе в дясно и (0, 1) - последен пиксел от текстурата горе в ляво.

Пример:

1. **glBegin(GL_QUADS);**
2. **glTexCoord2f (0.0, 0.0); glVertex3f (-1.0 , -1.0f, 0.0);** // долен ляв ъгъл
3. **glTexCoord2f (1.0, 0.0); glVertex3f (1.0, -1.0f, 0.0);** // долен десен ъгъл
4. **glTexCoord2f (1.0, 1.0); glVertex3f (1.0, 1.0f, 0.0);** // горен десен ъгъл
5. **glTexCoord2f (0.0, 1.0); glVertex3f (1.0, 1.0f, 0.0);** // горен ляв ъгъл
6. **glEnd();**

Ако квадрата беше няколко пъти по - голям, отново може да се ползват същите координати за текстурата, но тя щеше да бъде уголемена, което има своите отрицателни последици за качеството на текстурата. Ако пък за правоъгълник, чиято височина е 1/2 от ширината му и отново се използват същите текстурни координати, текстурата ще бъде сбита откъм височина, което отново може да има своите негативни страни за качеството. Ако се вземат за пример първо правоъгълника чиято височина е 1/2 от ширината то текстурните координати трябва да са съответно (0, 0) - първи пиксел от текстурата долу в ляво, (1, 0) - последен пиксел от текстурата долу в дясно, (1, 0.5) - пиксел най-вдясно от средата на текстурата и (0, 0.5) - пиксел най-вляво от средата на текстурата . Това ще доведе до наслагване само на половината от текстурата и така няма да има "сбиване" до размерите на правоъгълника.

Ако се вземе за пример квадрата, който е приблизително 4 пъти по-голям от текстурата, координатите на текстурата могат да са съответно (0, 0) , (2, 0) , (2, 2) и (0, 2). Като се има предвид че цифрата 1 съответства на краен пиксел от текстурата по x или y, то тези координати не могат да бъдат само от една текстурна единица, или в случая квадрата ще се покрие от четири отделни копия на текстурата. Координати от рода на (0, 0) , (10, 0) , (10, 10) и (0, 10) ще доведат до покриването на квадрата със 100 текстурни единици и т.н. Функцията **glTexCoord()** има доста варианти и някои от тях приемат до 4 параметъра за координатите на текстурата, включително и указател към масив със съхранените координати.

В случай, че трябва да се определят координатите на някой от готовите обекти създадени с библиотеките **GLAUX**, **GLUT** или **GLU** (релеф и квадратични обекти), като тези обекти се състоят примерно от много на брой полигони, а и няма пряк достъп до върховете на обекта, за да се използва **glTexCoord*()** се използва автоматичното определяне на координатите за дадена текстура върху някой обект. То върши автоматично работа по определяне на координатите, използвайки множество текстурни единици за по-големите обекти и предлагащо някои ефекти. Ръчното определяне има своето място, когато ефектите на автоматичното не задоволяват потребителя. Включването на автоматично генериране на координати трябва да подадете на функцията **glEnable()** една от константите **GL_TEXTURE_GEN_S**, **GL_TEXTURE_GEN_T**, **GL_TEXTURE_GEN_R**, или **GL_TEXTURE_GEN_Q**. Първите две, отговарят на генериране на координати съответно по x и y, а това е достатъчно в двумерния случай. След като включихте автоматичното генериране идва ред на функцията **glTexGen*()**, която определя начина на генериране на текстурните координати :

void glTexGen{ifd}(GLenum coord, GLenum pname, GLdouble param) - функцията приема 3 аргумента. Първият може да е една от константите **GL_S**, **GL_T**, **GL_R** или **GL_Q**. **GL_S** и **GL_T** указват изчисляване на текстурните координати съответно по x и y. Трябва да се определи начин на изчисляване на координатите както за x така и за y. Вторият аргумент трябва да е **GL_TEXTURE_GEN_MODE**, а третият може да бъде един от следните :

GL_OBJECT_LINEAR - текстурната единица/и се наслаждава по обекта.

GL_EYE_LINEAR - текстурата единица/и не залепва върху повърхността на обекта, а просто се плъзга по него в зависимост от неговото движение/деформации. Така може да симулирате вода с вълни, ако има разбира се повърхност под текстурата, която да се деформира, образувайки вълните.

GL_SPHERE_MAP - текстурата се наслаждава по обекта като отражение. Използван за сфера, която да рефлектира околната среда.

Пример за включване на автоматичното определяне на текстурните координати:

1. // Включване автоматично определяне на текстурните координати
2. **glEnable(GL_TEXTURE_GEN_S);**
3. **glEnable(GL_TEXTURE_GEN_T);**
4. // задаване начина на разпростиране на текстура - да се залепи за обекта
5. **glTexGeni(GL_S, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);**
6. **glTexGeni(GL_T, GL_TEXTURE_GEN_MODE, GL_OBJECT_LINEAR);**

За задаване на параметри на текстурата се използва функцията **glTexParameter*()**. Първата константа трябва да е **GL_TEXTURE_2D** за двуизмерни текстури, но за втора константа освен **GL_TEXTURE_MIN_FILTER** и **GL_TEXTURE_MAG_FILTER**, могат да се

подадат още **GL_TEXTURE_WRAP_S** и **GL_TEXTURE_WRAP_T**, които контролират текстурните координати по x или y на множеството текстурни единици върху дадена повърхност, когато има такива разбира се. Когато се включат някоя от тях третия аргумент трябва да бъде **GL_REPEAT** или **GL_CLAMP**. Ако се избере **GL_REPEAT** и за двете, няма да има разлика върху областта, в чието запълване участват 4 текстури, но при избор на **GL_CLAMP**, това ще доведе до размножаване на крайните пиксели от първата текстура надясно и нагоре т.е. генерират се координати само по x и само по y, с изключение на първоначалната текстура. Ако пък се избере **GL_REPEAT** за **GL_TEXTURE_WRAP_S** и **GL_CLAMP** за **GL_TEXTURE_WRAP_T**, ще се получат правилни текстури по дължина (т.е. долните две), но по височина (на мястото на горната двойка текстури) ще са се размножили крайните, горни пиксели от текстурите от първи ред. За получаване на контурна рамка на текстурата трябва да се извика функцията **glTexParameterfv()**, с аргументи съответно константите **GL_TEXTURE_2D**, **GL_TEXTURE_BORDER** и масив от тип **float** с **RGB** стойностите, определящи цвета на рамката.

Texture матрица **GL_TEXTURE**

Тази матрица манипулира самите текстури. Може да се ползват и функциите **glRotate*()** и **glTranslate*()**, както и **glPushMatrix()** **glPopMatrix()**. В случай, че трябва да се направи реалистично небе тези функции могат да се използват по следния начин.

Пример: - След изчистване на буферите с извикана функцията **glLoadIdentity()** за **Modelview** матрицата в рендерираща функция може да се включи успешно промяна на **TEXTURE** матрицата по следния начин:

```

1.  glMatrixMode( GL_TEXTURE );
2.  // Трябва да се извика glLoadIdentity( )
3.  // за да се включат първоначалната TEXTURE матрица
4.  glLoadIdentity( );
5.  /* Следва манипулацията,
6.  //glRotate*( ) и завъртане по z се постигат най - добри
7.  //резултати т.е. текстурата наистина се върти, докато при
8.  //x и y наблюдаваме удължаване и свиване на текстурата.
9.  //Манипулиране на текстурата, а не 3D обект,
10. // //При glTranslate*( ) трансформацията е по x или y е просто
11. //движение на текстурата по обекта, докато при z координатата
12. //Използване само glRotate*( )
13. //с изменение по z, ако искате да правите нормално небе или море. */
14. //glRotatef( angle , 0.0 , 0.0 , 1.0 );
15. // Превключване на Modelview матрица
16. // за да се създаде самото небе, което ще е един квадрат
17. glMatrixMode( GL_MODELVIEW );
18. // Свързване на текстурата за небето, която предварително
19. // е построена
20. glBindTexture( GL_TEXTURE_2D , clouds[0] );
21. // Изграждане на квадрата и определяне на текстурните
22. // координати за всеки връх
23. glBegin(GL_QUADS);
24. glTexCoord2f( 0.0 , 0.0 ); glVertex3f( 25.0f , 3.0f, 50.0 );
25. glTexCoord2f( 1.0 , 0.0 ); glVertex3f( -25.0f , 3.0f, 50.0 );
26. glTexCoord2f( 1.0 , 1.0 ); glVertex3f( -25.0f , 3.0f, -50.0 );
27. glTexCoord2f( 0.0 , 1.0 ); glVertex3f( 25.0f , 3.0f, -50.0 );
28. glEnd();
29. //промяна на TEXTURE матрицата се отразява на
30. //всички текстури, а ако трябва единствено да се манипулира
31. //текстурата на квадрата-небе, то трябва да се включи отново TEXTURE матрицата
32. //и да се зареди първоначалната TEXTURE матрица. */
33. glMatrixMode( GL_TEXTURE );
34. glLoadIdentity();
35. // Зареждаме отново Modelview матрицата за да се построиат

```


36. // останалите обекти от сцената, ако има такива

37. `glMatrixMode( GL_MODELVIEW );`

10.5 OPENGL - ЗАМЪГЛЯВАНИЯ

OpenGL предоставя функцията **glFog()**, с която може да създадете съвсем реалистична мъгла в триизмерна графика. Самата функция има четири разновидности, всяко от които си има определено предназначение :

void glFogf(GLenum pname, GLfloat param);

void glFogi(GLenum pname, GLint param);

void glFogfv(GLenum pname, const GLfloat *param);

void glFogiv(GLenum pname, const GLint *param);

За да се укаже, че ще се използва замъгляване трябва да:

- 1) зададе на функцията **glEnable()** аргумента **GL_FOG**.
- 2) След това е необходимо да укажете какъв тип мъгла ще се използва. За целта е нужно да се извика една от първите два вида на функцията **glFog*()**, като се подадете като първи аргумент **GL_FOG_MODE**, а за втори един от тези :**GL_LINEAR** - използва се за стандартни замъглявания (най-качествен режим на замъгляване) **GL_EXP** - също като **GL_LINEAR**, с известна разлика, **GL_EXP2** - използва се за силни замъглявания.
- 3) Следва определянето на цвета на мъглата, което най-добре може да се извърши с функцията **glFogfv()**, където за първи аргумент подадете **GL_FOG_COLOR**, а за втори - името на **float** масива в който са съхранени **RGBA** цвета (както при динамичната светлина).

Ако се използва мъгла тип **GL_LINEAR** е нужно да се определи на колко единици от позицията на камерата да започва мъглата и след колко единици обектите да са напълно скрити в нея. Това става като се подаде за първи аргумент на функцията **glFogf() / glFogi()** съответно аргументите **GL_FOG_START** или **GL_FOG_END**, а за втори аргумент на функцията определената числова стойност. Например : **glFogi(GL_FOG_START, 10)**. Добре е мъглата да започва винаги на няколко единици пред камерата, защото така се постигат по-добри резултати както във визуално качество, така и в производителност от графичния ускорител.

Ако се използват **GL_EXP** или **GL_EXP2** тип мъгла е нужно да се определят плътност за мъглата. За да се направи това е достатъчно да подадете **GL_FOG_DENSITY** като първи аргумент на функцията **glFogf()**, а за втори аргумент - стойност в интервала [0.0 - 1.0] , като по-голямо число води до по-силно замъгляване. Ако не се определи плътност на мъглата, **OpenGL** приема по подразбиране числото 1.0

Самото замъгляване се отразява само на цвета на обектите, затова е нужно в началото на рендериращата функция след изчистването на буферите, да се запълни цветовия буфер с цвета на мъглата или по-просто казано цвета на фона трябва да е еднакъв с цвета на мъглата. Цветовият буфер се запълва с функцията **glClearColor()**.

10.6 OPENGL - ПОЛУПРОЗРАЧНОСТ

В примерната графика голяма част от ефектите са реализирани благодарение на полу-прозрачността. Четвъртият компонент от **RGBA** режима се използва от **OpenGL** за установяване на степен на прозрачност. За да активирате обаче т.н. **Alpha Blending** не е достатъчно само да се зададе четвъртият аргумент във функцията **glColor4f**. Има някои важни особености.

Най-напред трябва да се активира полу-прозрачността с функцията **glEnable()** с аргумент **GL_BLEND**. След това трябва да се определи начина, по който се смесват цветовете на предния и задния обект, в което всъщност се състои полу-прозрачността. Формулата по която се изчислява полу-прозрачността в **OpenGL**:

(Rs*Sr+Rd*Dr, Gs*Sg+Gd*Dg, Bs*Sb+Bd*Db, As*Sa+Ad*Da) , където **Rs,Gs,Bs** и **As** са **RGBA** стойностите на предния обект, които задавате с **glColor4f()** , а **Rd,Gd,Bd** и **Ad** съответно **RGBA** стойностите на задния обект.

Sr,Sg,Sb,Sa и **Dr,Dg,Db,Da** са т.н. фактори на полу-прозрачността, от които зависи как ще се смесят цветовете на двата обекта, за да се постигне полупрозрачност. Благодарение на

тях може да се възпроизведе, най-различни ефекти на полу-прозрачност. Първата поредица от четири аргумента се отнася за предния обект, а втората за задния обект. Това се извършва с функцията:

void glBlendFunc (GLenum sfactor, GLenum dfactor), която приема два аргумента, от които зависи какви стойности ще получат факторите на полу-прозрачност съответно за предния и задния обект. Възможните аргументи са:

GL_ZERO - цвета на обекта не се взима предвид. За стойности на фактора на полу-прозрачност, на който и да е от двата обекта се предават стойностите (0, 0, 0, 0). Може да се използва както за преден, така и за заден обект.

GL_ONE - използва се цвета на обекта. За стойности на фактора на полупрозрачност, на който и да е от двата обекта се предават стойностите (1, 1, 1, 1). Също няма значение за кой от двата обекта определят фактор на полу-прозрачност.

GL_DST_COLOR - цвета на обекта се умножава по този на задния обект. Може да се използва единствено за предния обект, т.е. само като първи аргумент на функцията. За фактор на полу-прозрачност на предния обект се предават стойностите **Rd,Gd,Bd** и **Ad** т.е. **RGBA** цвета на задния обект.

GL_SRC_COLOR - цвета на обекта се умножава по този на предния обект. Може да се използва единствено за задния обект, т.е. само като втори аргумент на функцията. За фактор на полу-прозрачност на задния обект се предават стойностите **Rs,Gs,Bs** и **As** т.е. **RGBA** цвета на предния обект.

GL_ONE_MINUS_DST_COLOR - цвета на обекта се умножава с (1, 1, 1, 1 минус цвета на задния обект). Може да се използва само за предния обект (**sfactor**) и реално фактора за полу-прозрачност на предния обект получава стойности (1, 1, 1, 1) - (**Rd, Gd, Bd, Ad**), където последните четири както виждате представляват цвета на задния обект.

GL_ONE_MINUS_SRC_ALPHA - цвета на обекта се умножава с (1, 1, 1, 1 минус цвета на предния обект). Може да се използва само за аргумент на **dfactor**. Съответно тук за фактор на полу-прозрачност задния обект получава стойностите (1, 1, 1, 1) - (**Rs, Gs, Bs, As**).

GL_SRC_ALPHA - цвета на който и да е от двата обекта(може да го използвате както за **sfactor** така и за **dfactor**) се умножава с четвъртата стойност (**Alpha**) на предния обект стойност т.е. за фактори на полу-прозрачност обекта получава стойностите (**As, As, As, As**).

GL_DST_ALPHA - абсолютно същото с изключение на това, че за фактори на полу-прозрачност обекта получава четвъртата **RGBA** стойност от цвета на задния обект - (**Ad, Ad, Ad, Ad**).

GL_ONE_MINUS_DST_ALPHA - цвета на обекта се умножава с (1, 1, 1, 1 минус четвъртата **RGBA** стойност на задния обект) т.е. за фактор на полу-прозрачност се получават стойностите (1, 1, 1, 1) - (**Ad, Ad, Ad, Ad**) Този аргумент може да бъде използван във функцията **glBlendFunc()** както за предния, така и за задния обект.

GL_ONE_MINUS_SRC_ALPHA - същото като по горе с изключение на това че се използва **alpha** от предния обект т.е. за фактор на полу-прозрачност се приемат стойностите (1, 1, 1, 1) - (**As, As, As, As**).

Най-често използваните фактори за полу-прозрачност са **GL_SRC_ALPHA** за предния обект и **GL_ONE_MINUS_SRC_ALPHA** за задния обект. Важно е да се отбележи че ако се използва полу-прозрачност трябва да се изрисува първо задния обект и след това този отпред (за който е разрешена полу-прозрачността

Технология Antialiasing

Тя предполага,че при изобразяването на линия се използват по-голям брой пиксели като различното е, че за крайните пикселни единици се задава полу-прозрачност. Така окото ни не може да разграничи ясно чупките на линията и ефекта се получава - имаме гладка, не-нащърбена линия. Пример как да се изглаждат точки (изглеждат "по-кръгли"), прости линии или обекти изобразени с помощта на множество линии или точки. Изглаждането на краищата на запълнени обекти е доста по-трудно и няма да бъде разгледано, а и вместо него може да бъде използвано пълно-екранно изглаждане на всичко по сцената. За да се изгладят линиите или да се направят точките по-заоблени, е нужно да подадете на функцията **glEnable()** за аргумент **GL_LINE_SMOOTH** или **GL_POINT_SMOOTH**. След това е нужно да се разреши полу-прозрачност, като се подадете за аргумент на същата функция константата **GL_BLEND** и след това да се определи режим на смесване на цветовете на обектите, като се подаде на функцията **glBlendFunc()** аргументите **GL_SRC_ALPHA** и

GL_ONE_MINUS_SRC_ALPHA, съответно за фактори на полу-прозрачност на предния и задния обект.